RAIT CTF Report

Team Name: THE EAGLE EYE
CTF Name: RAIT-CTF
Date: 10 Jan 2026

Team Members

- Harsh
- Sahil
- Gyan
- Aakash

Challenges Solved

- Reverse 1 - Missile Evade
- Reverse 2 - Laser Control
- Web 1 - The Blueprint Recon
- Crypto 1 - Damaged Voice Recording

Reverse 1 - Missile Evade

By decompiling the executable using ghidra, we find that the program compares the transformed input with certain values which we can extract and and bruteforce the values for.

```c
// extflag.c

#include <stdint.h>
#include <stdio.h>

/* 8-bit rotate helpers (exactly match the binary semantics) */
static inline uint8_t rol(uint8_t x, uint8_t n) {
    return (uint8_t)((x << n) | (x >> (8 - n)));
}

static inline uint8_t ror(uint8_t x, uint8_t n) {
    return (uint8_t)((x >> n) | (x << (8 - n)));
}

uint8_t transform(int32_t arg1, uint8_t arg2)
{
    uint8_t v = arg2;

    switch (arg1)
    {
        case 0:  v = ror(ror(v,5) + 0x6d,6) + 0x64; break;
        case 1:  v = rol(v + 0x56,7); break;
        case 2:  v = rol(ror(rol(v,1),6) + 0x1c,4) + 0x56; break;
        case 3:  v = (((v ^ 0xd6) + 0x67) ^ 0x76) - 0x3e; break;
        case 4:  v = (ror(ror(v + 0x52,4),3) - 0x73) ^ 0xa4; break;
        case 5:  v = ror(rol(ror(ror(v,5),7) ^ 0x13,6),1); break;
        case 6:  v = ror((((v ^ 0xa3) + 0x33) ^ 0x49) + 0x74,3); break;
        case 7:  v = (rol(ror(v ^ 0xcc,3),5) - 0x73) ^ 0xc7; break;
        case 8:  v = ror(ror(v,6) + 0x4b,5); break;
        case 9:  v = ror(rol((v - 0x39) ^ 0xc0,6),4) ^ 0xcf; break;
        case 10: v = ror(ror(ror(ror(v,4),7) + 0x43,7),6); break;
        case 11: v = ((((v - 0x44) ^ 0xb6) - 0x0d) ^ 0xe8) + 0x51; break;
        case 12: v = ((rol(v ^ 0x32,5) + 0x1f) ^ 0x8e) + 0x6e; break;
        case 13: v = rol(((v + 0x64) ^ 0x6f) + 0x51,3); break;
        case 14: v = ror(rol(v,1) + 0x5b,3) - 0x52; break;
        case 15: v = rol(rol(v,5),5) + 0x29; break;
        case 16: v = rol(ror(ror(v,2),4) - 0x48,5) - 0x0b; break;
        case 17: v = rol(ror(rol(ror(v ^ 0x8b,2),3),3),7); break;
        case 18: v = rol(rol(rol(v - 0x1f,5),6),2); break;
        case 19: v = (ror(v,7) - 0x61) ^ 0x2e; break;
        case 20: v = ror(ror((v ^ 0xd8) - 0x7e,7),4) + 0x0a; break;
        case 21: v = (rol(rol(ror(v,5),2),1) + 0x4e) ^ 0x7e; break;
        case 22: v = ((ror(v + 0x5c,3) + 0x6b) ^ 0xaf) - 0x7e; break;
        case 23: v = ror(ror(v,6) ^ 0xa5,4) + 0x21; break;
        case 24: v = (rol(ror(rol(v,1),4),6) ^ 0x2d) + 0x2a; break;
```

```
        case 25: v = rol(rol((v + 0x67) ^ 0xb6,4),6); break;
        case 26: v = (rol(v + 0x38,3) ^ 0xc9) + 0x15; break;
        case 27: v = ((v ^ 0xe9) - 0x5d) ^ 0x29; break;
        case 28: v = (ror(rol(ror(v,6),5),2) ^ 0x6f) - 0x51; break;
        case 29: v = (rol(ror(rol(v,3),4),6) + 0x13) ^ 0x12; break;
        case 30: v = (rol(v - 0x38,1) ^ 0xf0) + 0x10; break;
        case 31: v = ror(rol(ror(v,5) ^ 0xd9,3),5) - 0x28; break;
        case 32: v = (rol(rol(v ^ 0xd9,3),3) ^ 0xc5) - 0x04; break;
        case 33: v = rol((ror(v,1) ^ 0x11) + 0x32,5); break;
    }

    return v;
}

int main()
{
    uint8_t cipher[34] = {
        0x63,0xcb,0xbc,0x61,0x2f,0xe4,0xfa,0x70,
        0xc1,0x2f,0x8e,0xb5,0xb0,0x40,0x65,0xde,
        0x85,0x9a,0xaa,0xd1,0x2b,0x16,0x35,0x68,
        0x01,0xb5,0xe1,0x54,0x33,0x70,0x11,0x82,
        0xe6,0x3c
    };

    char flag[35];

    for (int i = 0; i < 34; i++) {
        for (int ch = 32; ch <= 126; ch++) {
            if (transform(i, (uint8_t)ch) == cipher[i]) {
                flag[i] = (char)ch;
                break;
            }
        }
    }

    flag[34] = '\0';
    printf("Flag: %s\n", flag);
    return 0;
}
```

Compile and run

```
gcc extflag.c -o extflag
./extflag
```

Flag obtained:

```
RAIT-CTF{1ts_t1m3_t0_h1t_th3_z0ne}
```

Reverse 2 - Laser Control

Given the executable `chall_ENNXmMh`
The executable expects an integer argument.

When running

```
./chall_ENNXmMh n
```

Where n is an integer value.
It outputs

```
[*] Charging laser to power level n

[OK] Weapon remains operational.
```

The chargin of the laser takes more time for greater values of n

Inspecting strings in the executable

```
gyan@inspiron:~/attackbox$ strings ./chall_ENNXmMh
/lib64/ld-linux-x86-64.so.2
puts
putchar
fflush
usleep
stdout
atoi
__libc_start_main
__cxa_finalize
printf
libc.so.6
GLIBC_2.34
GLIBC_2.2.5
__gmon_start__
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
ATUSH
 <^v
[]A\
PTE1
=Yf
5Rf
u+UH
AUATU
~CE1
[]A\A]A^A_
[31m
[93m
Usage: ./chall <power_level>
[!] POWER OVERLOAD
[!] CRITICAL BEAM INSTABILITY
[CORE DUMP]
=== LASER GUN POWER MODULE ===
[*] Charging laser to power level %d
[!] PERMANENT LASER SHUTDOWN ENGAGED
[OK] Weapon remains operational.
;*3$"
GCC: (Debian 14.3.0-5) 14.3.0
.shstrtab
.note.gnu.property
.note.gnu.build-id
.interp
.gnu.hash
.dynsym
```

```
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.note.ABI-tag
.init_array
.fini_array
.dynamic
.got.plt
.data
.bss
.comment
```

We find `[!] PERMANENT LASER SHUTDOWN ENGAGED` which is probably displayed when we are able to successfully shut down the laser.

For $n > 10000$ or $n < 0$ the program prints

```
[!] POWER OVERLOAD
```

So we can make an assumption that there is a value $n \in \mathbb{Z} \mid 0 \leq n \leq 10000$. for which the laser is shut down.

We can attempt to find that value of n using brute force.

```
gyan@inspiron:~/attackbox$ for i in $(seq 0 10000); do   out=$(./chall_ENNXmMh $i);
```

However, we quickly run into a problem.
The program waits for the laser to charge up to the power level before displaying the output.
This makes it impractical to brute force the answer in any reasonable amount of time.

We can solve this by overriding the `usleep(useconds_t usec)` function that the program is probably using.

Confirm that the program is calling `usleep`

Let's use `gdb` to set a breakpoint on `usleep` function and see if the program breaks.

```
gyan@inspiron:~/attackbox$ gdb ./chall_ENNXmMh
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./chall_ENNXmMh...
(No debugging symbols found in ./chall_ENNXmMh)
(gdb) break usleep
Breakpoint 1 at 0x3c060
(gdb) run 100
Starting program: /home/gyan/attackbox/chall_ENNXmMh 100
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[*] Charging laser to power level 100


Breakpoint 1, usleep (useconds=3000) at ../sysdeps/posix/usleep.c:24
warning: 24        ../sysdeps/posix/usleep.c: No such file or directory
(gdb)
```

The breakpoint is reached. This confirms our assumption that the program is using the `usleep` function to wait for the laser to recharge.

Next, we need to override the usleep function to make sure that it exits immediately upon being called without waiting.

```c
// fastsleep.c
#include <unistd.h>

int usleep(useconds_t usec) {
  return 0;   // pretend we slept
}
```

Compile it as a shared library

```
gyan@inspiron:~/attackbox$ gcc -shared -fPIC fastsleep.c -o fastsleep.so
```

Check if it works

```
gyan@inspiron:~/attackbox$ LD_PRELOAD=./fastsleep.so ./chall_ENNXmMh 1000
[*] Charging laser to power level 1000

[OK] Weapon remains operational.
```

There is no waiting for the laser to recharge now.

Proceeding with our brute force approach. The process still takes about 10 minutes, although it is way more practical now.

```
gyan@inspiron:~/attackbox$ for i in $(seq 0 10000); do   out=$(LD_PRELOAD=./fastslee
FOUND: 6746
```

The laser shuts down for $n = 6747$

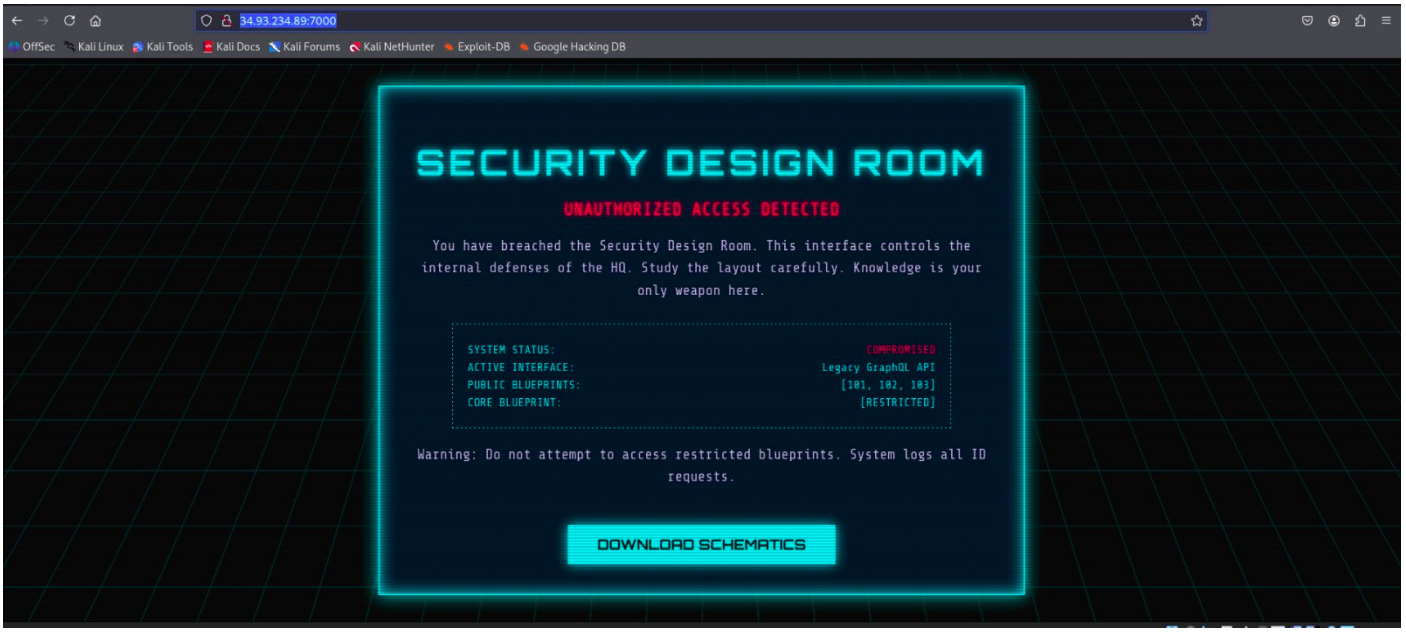
```
gyan@inspiron:~/attackbox$ LD_PRELOAD=./fastsleep.so ./chall_ENNXmMh 6746
[*] Charging laser to power level 6746

[!] CRITICAL BEAM INSTABILITY
[!] PERMANENT LASER SHUTDOWN ENGAGED
[CORE DUMP]
RAIT-CTF{D1d_y0u_adD_4_w4tch_p01nt_t0_1t?}
```

Flag obtained

```
RAIT-CTF{D1d_y0u_adD_4_w4tch_p01nt_t0_1t?}
```

Web 1 - The Blueprint Recon

Given ---> Access Here - http://34.93.234.89:7000.



1. Initial Reconnaissance & Error Analysis
   The challenge provided a web interface styled as a "Security Design Room." The HTML source explicitly mentioned a "Legacy GraphQL API".

Upon attempting to access the endpoint directly at /graphql via a browser (or a basic GET request), the server returned the following error:

req.body is not set; this probably means you forgot to set up the json middleware before the Apollo Server middleware.
Analysis: This error is specific to Apollo Server/Express environments. It indicates that the server is strictly expecting a POST request with a JSON body, but failed to parse it. This confirmed that the Content-Type: application/json header was missing from our initial requests.

2. API Mapping (Introspection)
   To fix the error, we switched to curl and manually injected the correct Content-Type header. With the connection established, we needed to understand the API structure. We sent a

standard GraphQL Introspection Query to list the available fields.

Request:

Bash

```bash
curl -X POST http://34.93.234.89:7000/graphql \
     -H "Content-Type: application/json" \
     -d '{"query": "{ __schema { queryType { fields { name args { name } } } } }"}'
```

Response:

JSON

```json
{
  "data": {
    "__schema": {
      "queryType": {
        "fields": [
          { "name": "publicBlueprints", "args": [] },
          { "name": "getBlueprint", "args": [{ "name": "id" }] }
        ]
      }
    }
  }
}
```

The introspection revealed a critical query: getBlueprint, which accepts an id argument. This suggested that we could retrieve specific blueprint details by providing an ID.

3. Exploitation (IDOR)
   The web page listed IDs 101, 102, and 103 as "Public Blueprints." We suspected an Insecure Direct Object Reference (IDOR) vulnerability where a hidden or "Restricted" blueprint existed at a different ID.

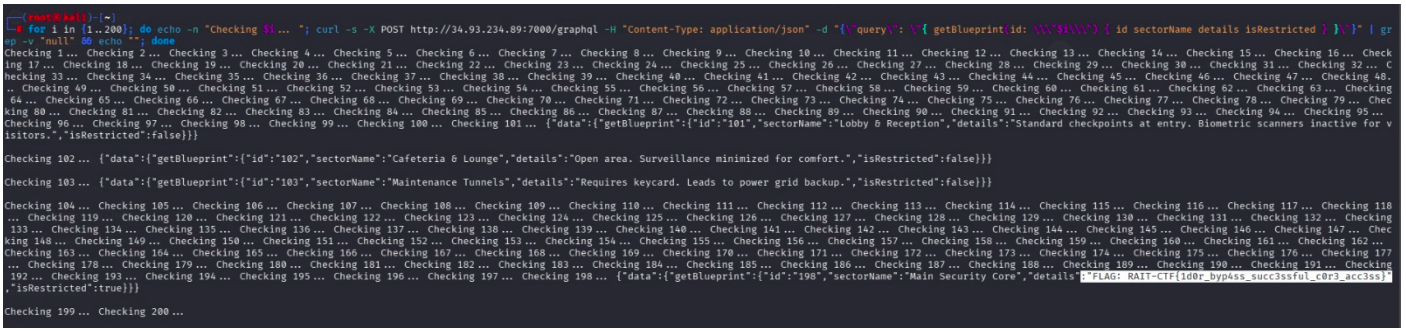
We wrote a bash loop to fuzz the id parameter, checking for any response that contained data.

The Fuzzing Logic:

Bash

```bash
for i in {1..200}; do
    curl -s -X POST http://34.93.234.89:7000/graphql \
    -H "Content-Type: application/json" \
    -d "{\"query\": \"{ getBlueprint(id: \\\"$i\\\") { id sectorName details } }\"}"
done
```

While IDs 101-103 returned standard lobby info, the script hit a unique response at ID 198.

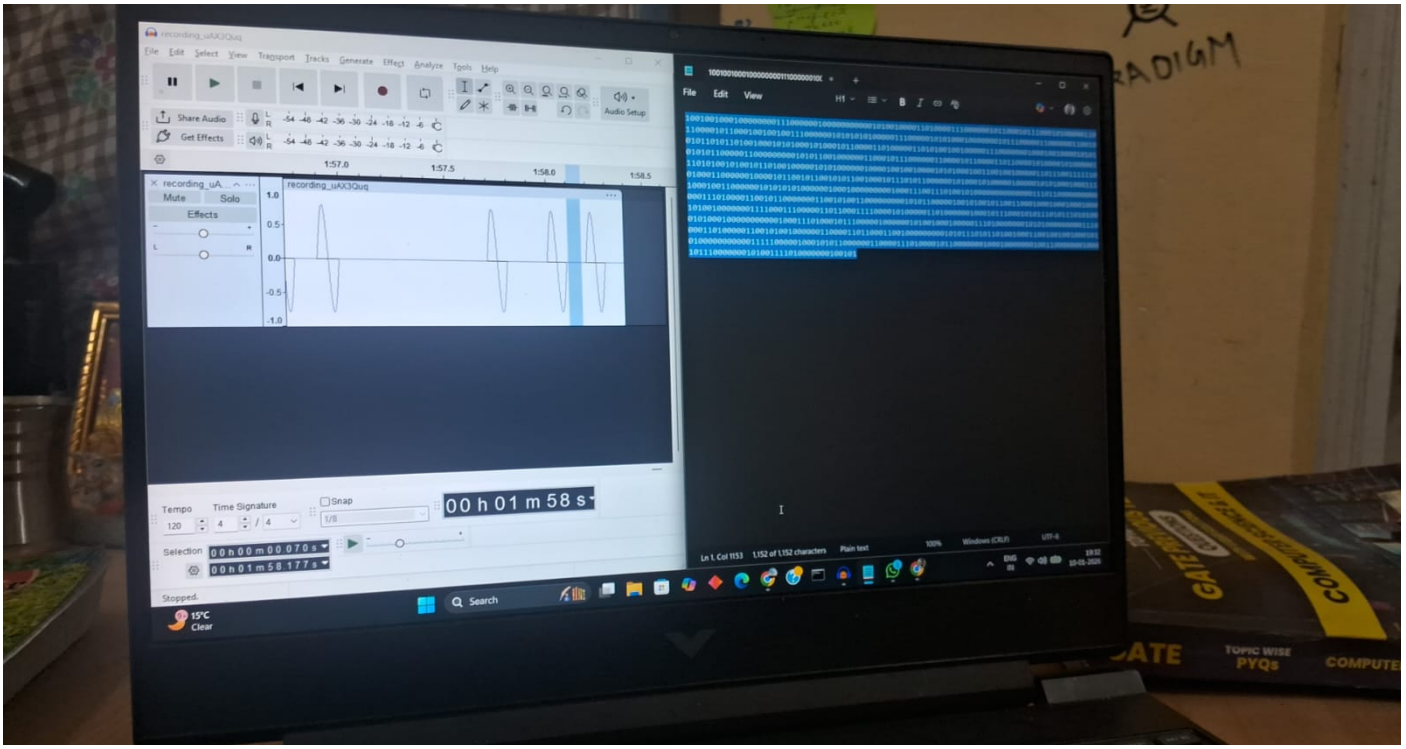


Flag obtained

```
RAIT-CTF{1d0r_byp4ss_succ3ssful_c0r3_acc3ss}
```

Crypto 1 - Damaged Voice Recording

In this challenge we used tools like Audacity and Binary Decoder, but while solving challenge flag was out on the discord



Flag obtained

```
RAIT-CTF{b10d1g174l_v1ru5_1n_th3_c0r3}
```